AI SRE · attendee workbook

What an AI SRE is, and what has to be true before you run one

This is the explanation of every idea the workshop uses, written once. You will learn what telemetry an agent can reason over and how a detector decides that something is wrong. You will also learn what separates a tool an agent may hold from one it may not, and where a person has to stand in the loop. The facilitator guide carries the timing and the commands for the day, and it points back at these ideas by name.

Read for	engineers and engineering leaders who are deciding how much an agent may do on their systems
Proven by	`make check` and `make e2e` in this repository, against a real model and a real browser
Printed by	`make book`, which writes the PDF and reads its own text layer back

Contents

01 **What are we building, and why?**
Overview

02 **What must the agent be able to see?**
Module one

03 **How do we say what good means?**
Reliability targets

04 **How does the agent know something is wrong?**
Module two

05 **Once it is wrong, where is the cause?**
Module three

06 **What may the agent actually do?**
Module four

07 **Where does the person come in?**
Module five

08 **How can this go wrong?**
Module six

09 **Where do these ideas come from?**
Grounding

10 **The words this workshop uses**
Recap

Overview

What are we building, and why?

Every incident starts with reading. We give the reading to an agent, and keep the decision with a person.

Signups fail at 2am. Who works out why?

Someone must find what changed before anyone can fix it. That search is the slow part.

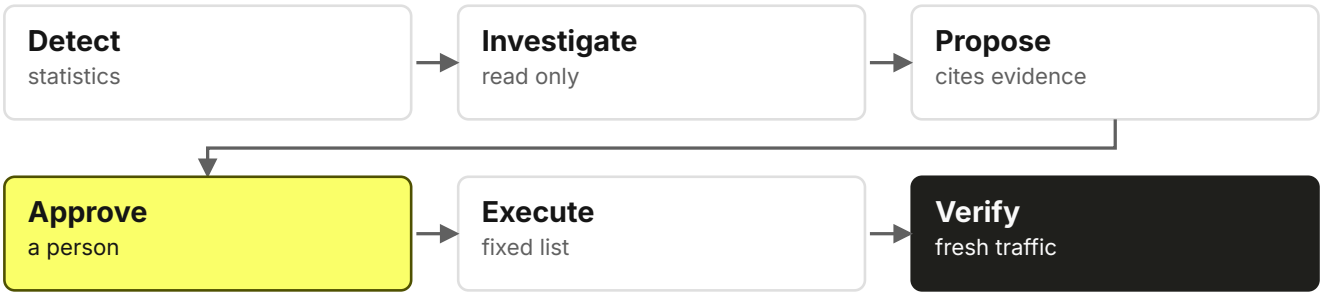


Figure 1 The agent does the reading. A person makes the call.

So an agent does the searching. It reads the data and writes down its best guess, with proof. A person reads the proof and decides.

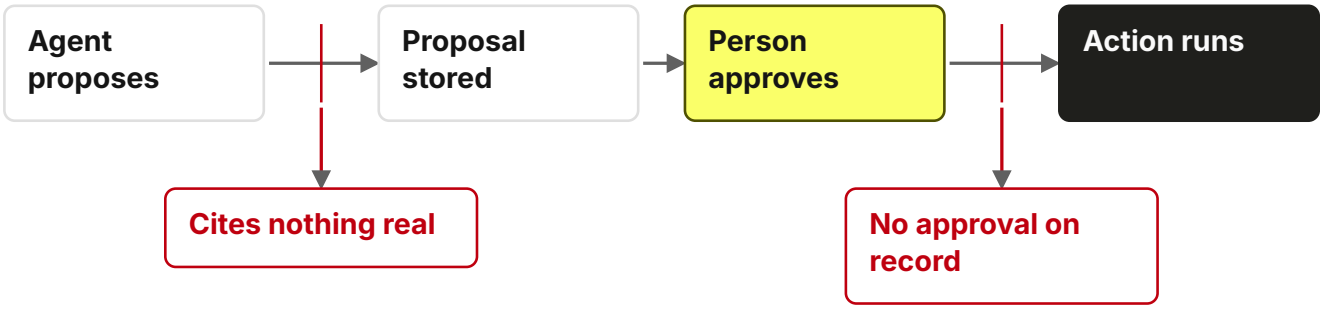


Figure 2 Both stops are the server saying no. A log line cannot argue with them.

But an agent can be wrong, and a prompt cannot stop it. So the server does the stopping. It turns away a guess that points at nothing, and it will not run anything until a person has said yes.

> **Next problem**
>
> **That is the job. But people will fear what this agent could become.**

What must this agent never become?

Someone always asks whether the model will take the site down. Better to answer before they ask.

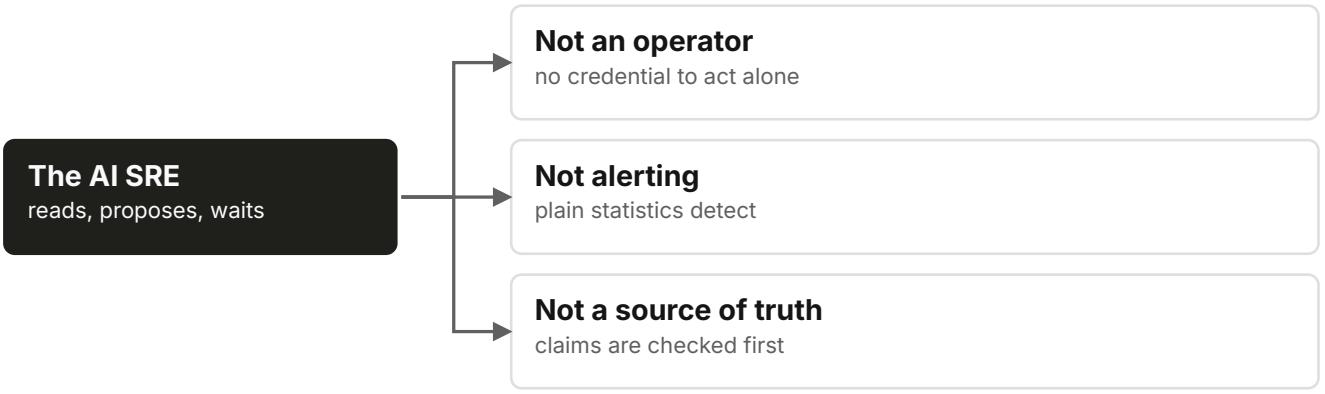


Figure 3 Each role is closed off by something you can test.

So we write a list of nots. Each one is a job people will hand it if nobody says no. Each one also becomes a test you can run.

> **Next problem**
>
> **Nots mean little without good data. First we must decide which signal to open.**

Module one

What must the agent be able to see?

Every failure in this chapter happens before a model is involved. If SQL cannot answer these questions, no agent can.

Which signal do I open first?

A customer says checkout was slow around noon. You keep logs, metrics and traces, and one of them answers this fastest.

Signal	Answers	Costs
Log	What went wrong, once	The most bytes
Metric	How much, how often	The least
Trace	Where the time went	A tree per request

Figure 4 Only a trace shows the path a request took.

So we keep all three. Metrics show that something changed. Logs show what broke. Traces show which step was slow.

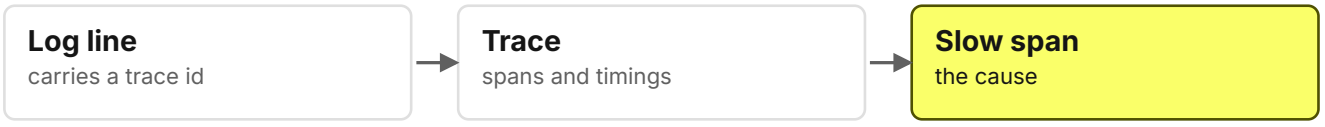


Figure 5 One store lets a question hop from a log line to the slow step.

Three tools means three tabs and a lot of guessing. So we store them side by side, and one query can hop between them.

```
SELECT
    table,
    sum(rows) AS rows,
    formatReadableSize(sum(data_compressed_bytes)) AS compressed,
    round(sum(data_uncompressed_bytes) / greatest(sum(data_compressed_bytes),
1), 1) AS ratio
FROM system.parts
WHERE database = 'default' AND active AND table LIKE 'otel_%'
GROUP BY table
ORDER BY rows DESC
LIMIT 20;
```

Run this on your own store. If a signal is missing, fix that first.

> **Next problem**
>
> **Having the logs is not enough. We still cannot say if one release caused the errors.**

Did the errors all come from one release?

Errors doubled at 3am. The answer depends on a choice someone made in a code review last spring.

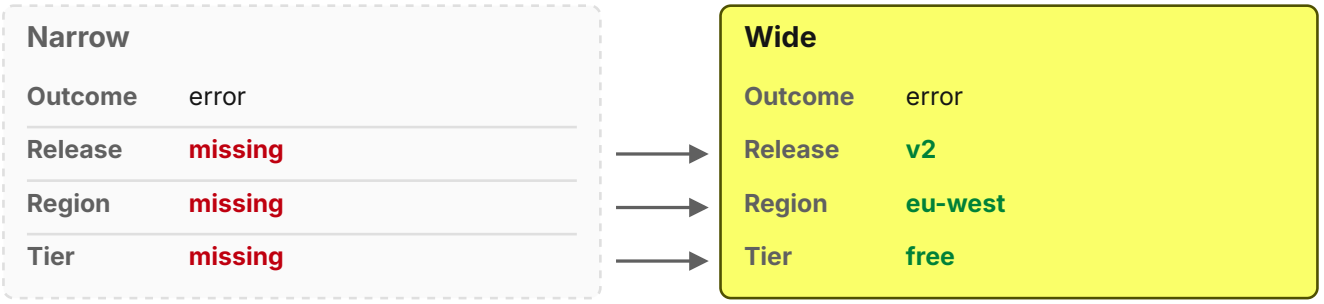


Figure 6 The same failed request, written twice. Only the wide one can be split by release later.

So we write one wide record for every request, with every fact we might want later. A fact nobody wrote down is gone, and nobody can add it at 3am.

```
SELECT
    ResourceAttributes['service.version'] AS release,
    min(Timestamp) AS first_seen,
    count() AS requests,
    countIf(StatusCode = 'Error') AS errors,
    round(errors / requests, 4) AS error_rate,
    round(quantile(0.95)(Duration) / 1e6, 1) AS p95_ms
FROM default.otel_traces
WHERE ServiceName = {service:String}
  AND SpanKind = 'Server'
  AND SpanName NOT IN ('GET /health')
  AND Timestamp > now() - toIntervalMinute({minutes:UInt32})
GROUP BY release
ORDER BY first_seen
LIMIT 20;
```

Try it on your own system. Split one minute of errors by release. If you cannot, fix that before anything else.

Next problem

But each extra fact costs money. We cannot afford to record everything.

Which facts can we afford to record?

Someone adds the user id to every metric so support can find one customer. That month the bill triples.

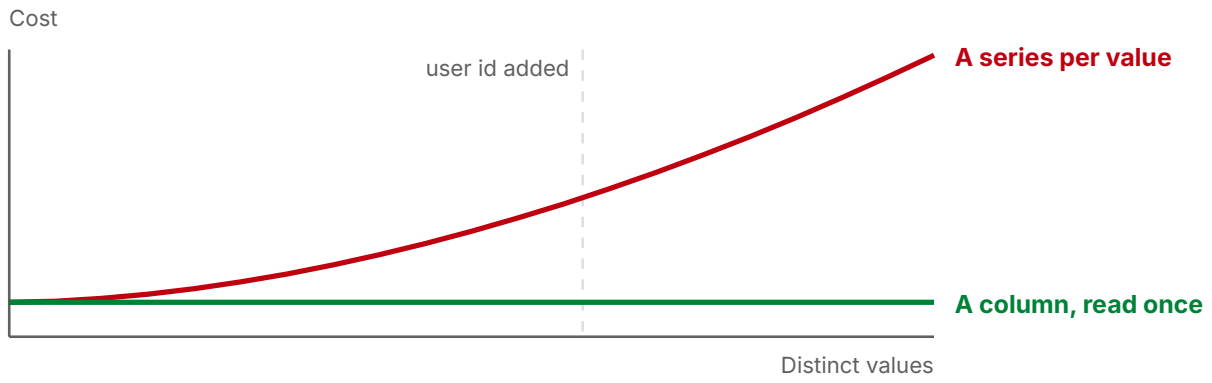


Figure 7 Cost climbs with the number of different values. The marked line is when someone added the user id.

Count how many different values a fact can take. Status code has a few. User id has millions, and a metric stores each one on its own.

Attribute	Distinct values	Good for
Status code	A handful	Charts
Release	Dozens	Blame
User id	Millions	One customer

Figure 8 The costliest fact is also the one that finds a single broken customer.

So do not drop the fact. Store it where many values are cheap. First ask whether your store bills per value or per column read.

> **Next problem**
>
> **Cost also pushes teams into sampling, and sampling has to throw something away.**

What does sampling throw away?

The dashboard says the slowest requests take about a second. A customer sends proof of a page that took forty. Both are true.



Figure 9 Slow requests are rare, so a fixed share drops them first.

Failures are rare, so a random sample loses most of them. What is left describes the requests that worked. So we keep every span, and the number we quote is the real one.

```
SELECT
    SpanName AS endpoint,
    count() AS requests,
    round(quantile(0.50)(Duration) / 1e6, 1) AS p50_ms,
    round(quantile(0.95)(Duration) / 1e6, 1) AS p95_ms,
    round(quantile(0.99)(Duration) / 1e6, 1) AS p99_ms,
    round(countIf(StatusCode = 'Error') / count(), 4) AS error_rate
FROM default.otel_traces
WHERE ServiceName = {service:String}
  AND SpanKind = 'Server'
  AND Timestamp > now() - toIntervalMinute({minutes:UInt32})
GROUP BY endpoint
ORDER BY requests DESC
LIMIT 50;
```

Ask what your sampler keeps when a request fails. If it keeps the same share as everything else, your numbers flatter you.

> **Next problem**
>
> **The data is now sound. But everything can look green while signups fail.**

Reliability targets

How do we say what good means?

These words come from Google's site reliability practice. The rest of the day uses them.

Everything is green but signups fail. What should we measure?

Processors are idle and memory is fine. Signups have failed for an hour, because nothing on the dashboard counts a signup.

Indicator	Good event	Valid event
Availability	Returned success	Any user request
Latency	Served under 300 ms	Any user request
Freshness	Data under a minute old	Any read

Figure 10 A ratio still makes sense when traffic doubles. A raw count does not.

So we measure what the user feels, as good events divided by valid events. This is called an indicator. Leave out events that prove nothing. A health check that always passes quietly improves every number.

```
SELECT
    toStartOfInterval(Timestamp, toIntervalSecond({bucket_s:UInt32}))
AS bucket,
    count() AS requests,
    countIf(StatusCode = 'Error') AS errors,
    round(errors / requests, 4) AS error_rate,
    round(quantile(0.95)(Duration) / 1e6, 1) AS p95_ms
FROM default.otel_traces
WHERE ServiceName = {service:String}
  AND SpanKind = 'Server'
  AND SpanName NOT IN ('GET /health')
  AND Timestamp > now() - toIntervalMinute({minutes:UInt32})
GROUP BY bucket
ORDER BY bucket
LIMIT 1000;
```

Write yours as one sentence with a top and a bottom. If you cannot, nobody can alert on it.

> **Next problem**
>
> **A number alone settles nothing. We must agree what is good enough for signups.**

How good is good enough for signups?

Product wants signups to be reliable. Engineering asks how reliable, over what time, measured how. The meeting ends there.

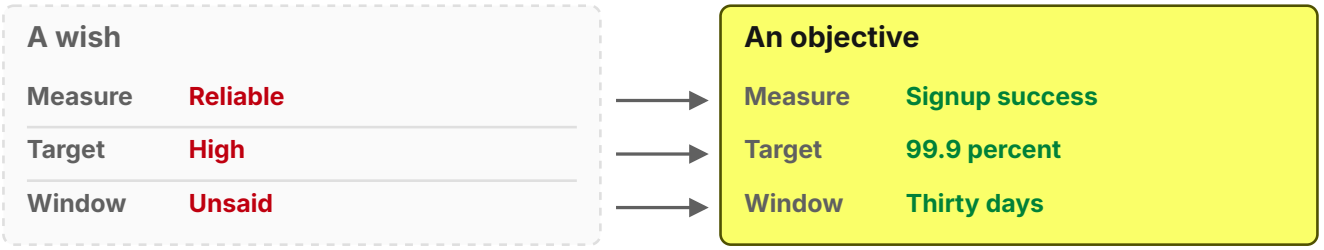


Figure 11 Only the right side can be argued with.

So we agree on a target: a number, a window, and the indicator it applies to. This is an objective. If missing it changes nothing, it is only decoration, so also write down what happens when it is missed.

> **Next problem**
>
> **A target draws a line, but says nothing of how much failure we may spend before it.**

How much failure can we spend?

A risky release is proposed on Tuesday. Someone asks if the team can afford it, and there is no number to answer with.



Figure 12 At 99.9 percent over thirty days, about forty three minutes of failure are allowed.

Take one minus the objective, times the window. That is the failure you are allowed, and it is called the error budget. When it is full, ship faster. When it is empty, stop and repair.

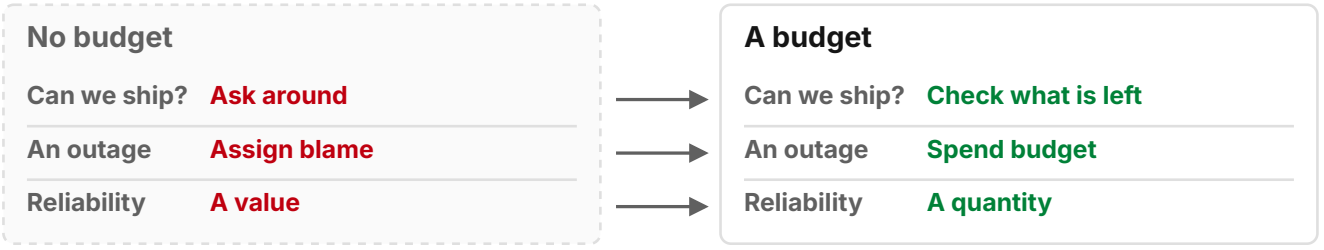


Figure 13 With a budget, the release talk becomes arithmetic that anyone can join.

Work out yours today. It is usually smaller than anyone expects.

> **Next problem**
>
> **A budget can drain slowly or fast. Only a fast drain is worth waking somebody for.**

When is it worth waking somebody up?

An alert fires for a spike that lasted forty seconds, so the person on call learns to swipe it away. The next one was real.

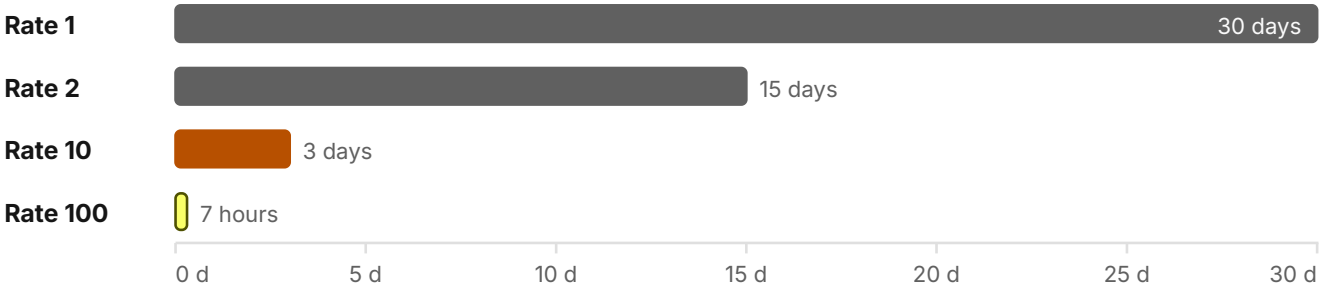


Figure 14 How long the whole budget lasts at each rate. Rate 1 is on target. Rate 100 is an outage.

A fixed threshold pages for every spike, or for none. So we page by how fast the budget is going. This is the burn rate. It tells the sleeper the one thing they need: how long until it is gone.

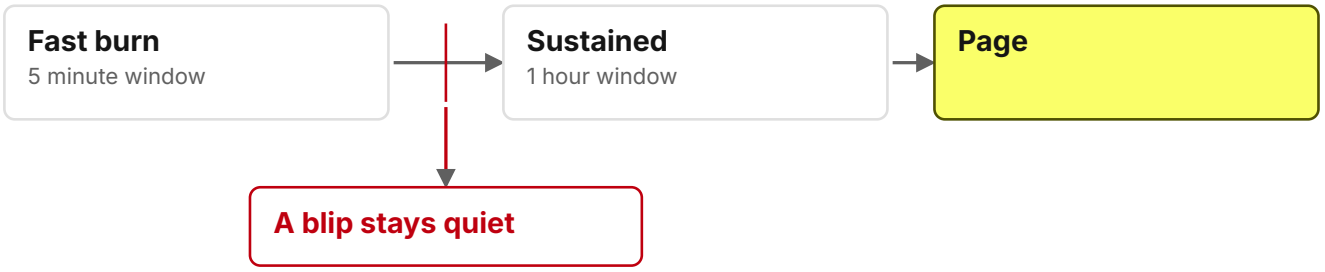


Figure 15 Both windows must agree, so a real burn pages and a blip stays quiet.

The lab uses a simpler cousin, because a workshop needs an incident within a minute. The idea still carries over. Compare with what normal looks like, never with a number typed once.

> **Next problem**
>
> **Both ideas compare against normal. Nobody has yet said what normal should look like.**

Module two

How does the agent know something

is wrong?

Detection is arithmetic over a baseline. The hard part is deciding what counts as normal.

What did normal look like?

A service fails one request in a hundred every day and nobody is paged. One afternoon it is one in ten. The detector must know the difference.

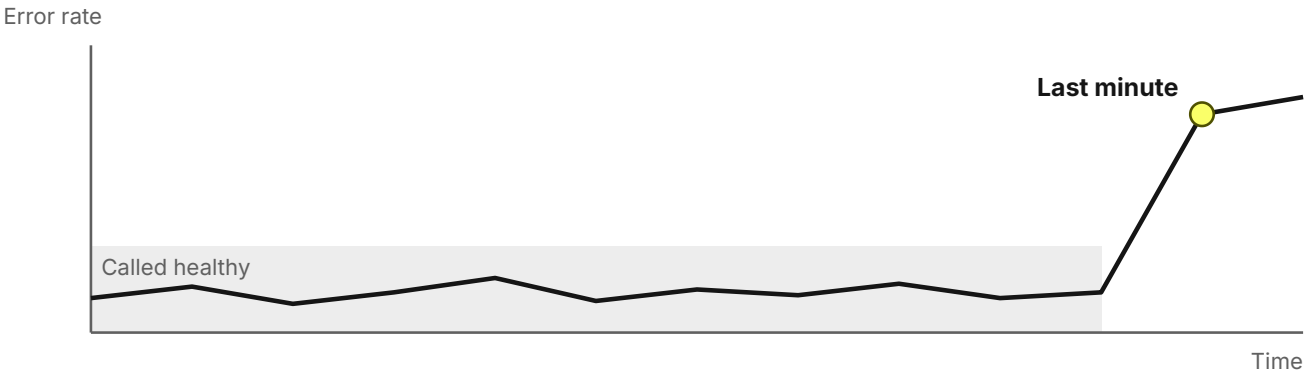


Figure 16 Every verdict is relative to the shaded stretch. Change it and every verdict changes.

So we pick a stretch of time and call it healthy. Too short, and a slow morning looks like an outage. Too long, and a service that has been getting worse for a week looks fine.

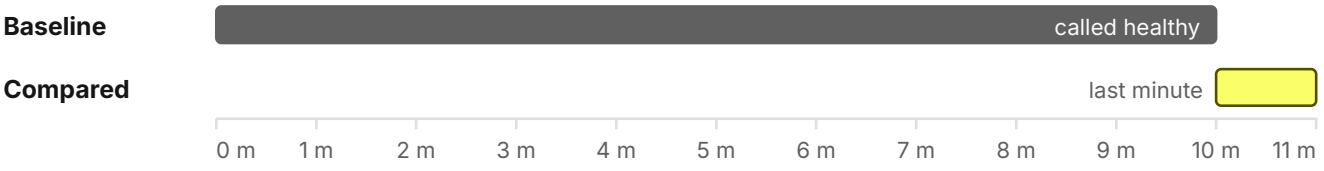


Figure 17 The last minute is judged against the ten before it, and is never part of them.

Decide what your detector may call healthy. Write it down where the next person will find it.

> **Next problem**
>
> **Normal is a starting point. Now we must choose how far from it an alert should fire.**

How far from normal should an alert fire?

Two minutes after the alert, someone asks why 0.041 opened an incident and 0.038 did not. Nobody chose either number.

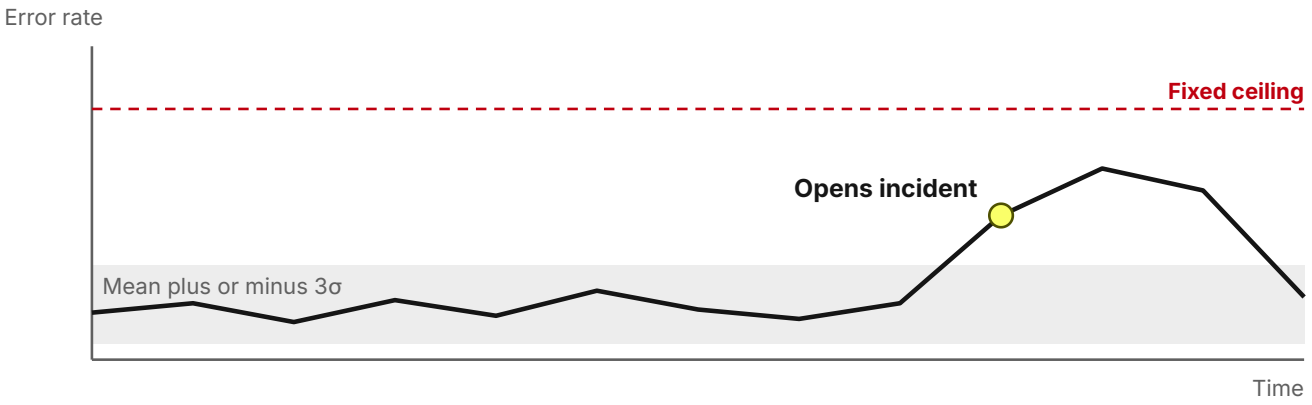


Figure 18 An illustration, not lab data. The dashed ceiling still catches a failure when no clean baseline exists yet.

So we draw a band around normal, three standard deviations wide. Ordinary minutes stay inside it, and a real jump leaves it. It is a trigger, not a promise of one in a thousand, because error rates are often not bell shaped.

```
WITH buckets AS (
    SELECT
        ServiceName AS service,
        toStartOfInterval(Timestamp, toIntervalSecond(10)) AS bucket,
        countIf(StatusCode = 'Error') / count() AS error_rate,
        quantile(0.95)(Duration) / 1e6 AS p95_ms,
        count() AS requests
    FROM default.otel_traces
    WHERE SpanKind = 'Server'
      AND SpanName NOT IN ('GET /health', '/')
      AND Timestamp > now() - toIntervalMinute({baseline_minutes:UInt32} + 1)
    GROUP BY service, bucket
)
SELECT
    service,
    round(avgIf(error_rate, bucket < now() - INTERVAL 60 SECOND), 4)
AS baseline_mean,
    round(stddevPopIf(error_rate, bucket < now() - INTERVAL 60 SECOND), 4)
AS baseline_sigma,
    round(baseline_mean + 3 * baseline_sigma, 4) AS bound,
    round(avgIf(error_rate, bucket >= now() - INTERVAL 60 SECOND), 4)
AS current_error_rate,
```

```
    round(avgIf(p95_ms, bucket >= now() - INTERVAL 60 SECOND), 1)
AS current_p95_ms,
    round(avgIf(p95_ms, bucket < now() - INTERVAL 60 SECOND), 1)
AS baseline_p95_ms,
    sumIf(requests, bucket >= now() - INTERVAL 60 SECOND) AS current_requests,
    current_error_rate > bound AND current_error_rate - baseline_mean >=
{min_delta:Float64} AS anomalous
FROM buckets
GROUP BY service
ORDER BY anomalous DESC, current_error_rate DESC
LIMIT 50;
```

Read the query as four choices: a mean, three deviations, a trailing window and a minimum change. Each one can be argued with.

> **Next problem**
>
> **The band is only as good as its baseline. A failure inside it can hide the next one.**

Does the last incident hide the next one?

You run the failure and the detector fires. Ten minutes later you run it again for a colleague, and nothing happens.

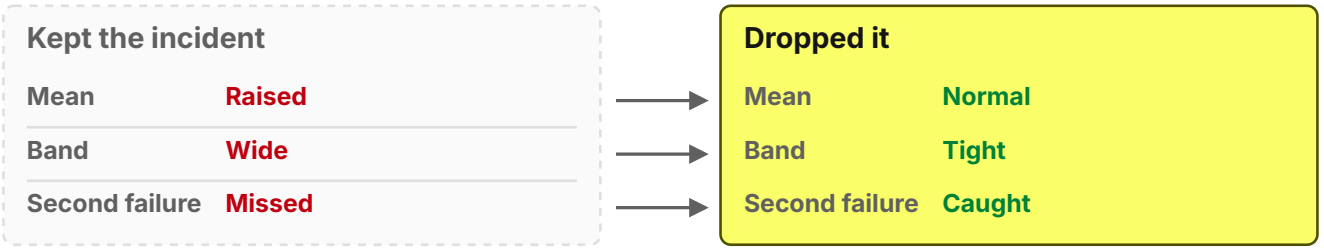


Figure 19 What one failure does to the next, depending on whether the baseline kept it.

The first failure raised the mean and widened the band, so the second one sits inside normal. So we drop past incidents from the baseline. Then every failure looks new.

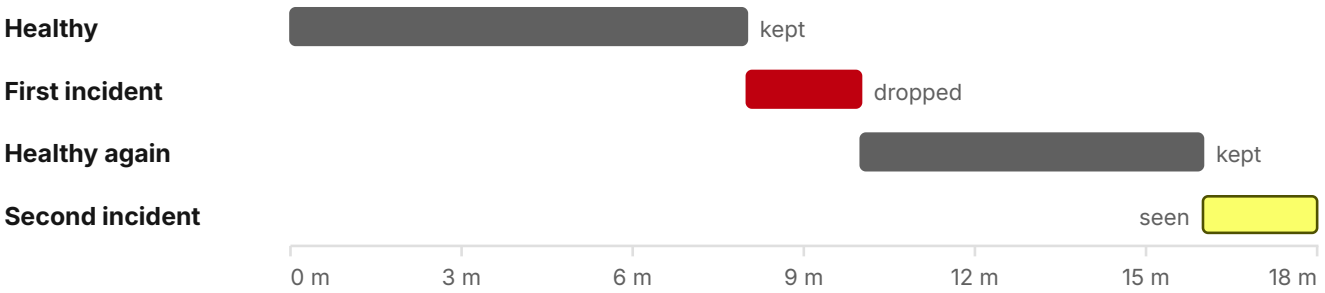


Figure 20 The dropped stretch is why the second failure still stands out.

Check your own detector. Fail the same thing twice and watch the second one.

> **Next problem**
>
> **A detector can be right and still be noisy. Take a page about a busy processor.**

Should we page for a processor at 85 percent?

The pager goes off at four because a processor touched 85 percent. Nothing was wrong. The person on call reads the next alert a little more slowly.

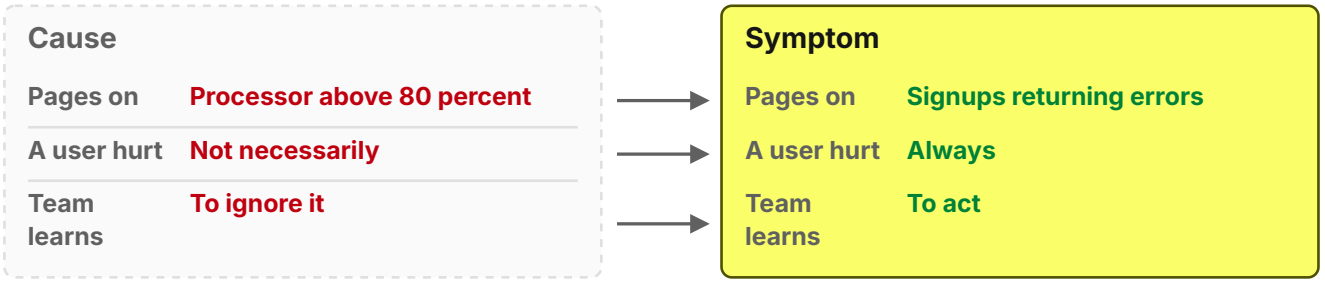


Figure 21 What each kind of alert teaches the person who receives it.

A cause that never reaches a user is a page that did not need to happen. Enough of them and a team stops trusting the pager. So we page only for a symptom a customer could screenshot.

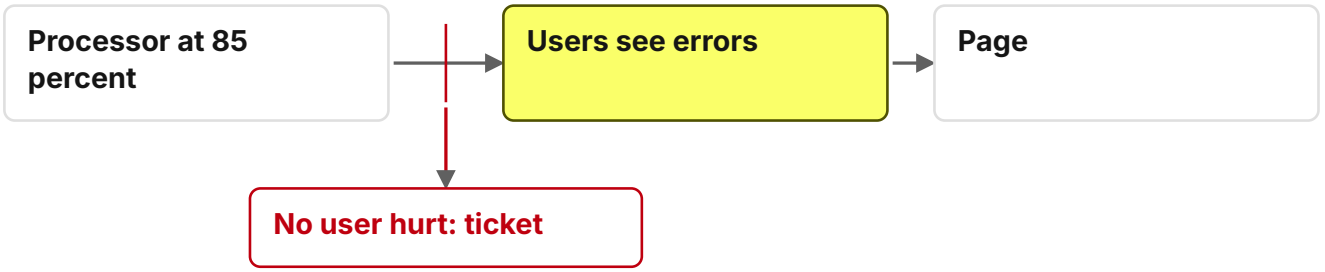


Figure 22 A page must pass this test. Cause alerts go to a ticket instead.

Keep the cause alerts. Send them to a ticket.

> **Next problem**
>
> **Now something is truly wrong. The failing requests must differ from the good ones somehow.**

Module three

Once it is wrong, where is the cause?

You know something fails. Now narrow it down to the one thing that sets the failing requests apart.

What is different about the failing requests?

Errors are up. Someone asks what the failed requests have in common, and the room goes quiet. There are forty attributes and no obvious place to start.

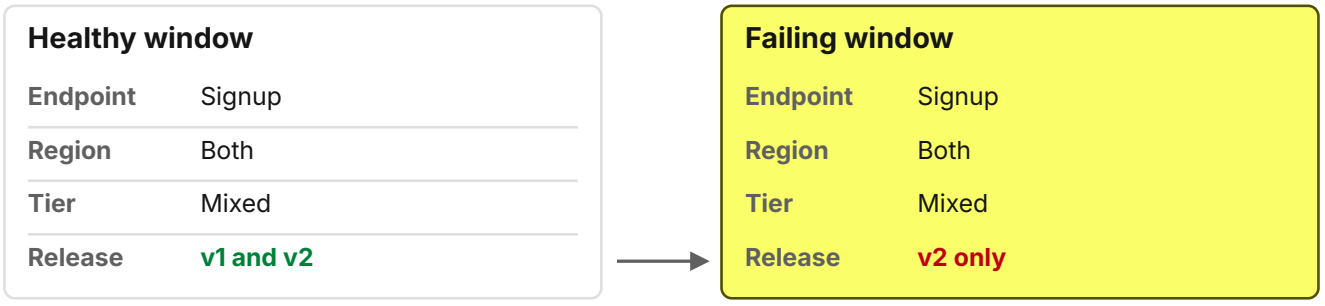


Figure 23 The attribute that changed between the windows rises to the top.

Left alone, an engineer picks the attribute they already suspect, and proves themselves right. So we compare every attribute at once between a healthy window and a failing one. The data picks.

```
WITH now() - toIntervalMinute({incident_minutes:UInt32}) AS incident_start
SELECT
    SpanName AS endpoint,
    ResourceAttributes['service.version'] AS release,
    StatusMessage AS status_message,
    countIf(Timestamp >= incident_start AND StatusCode = 'Error')
AS incident_errors,
    countIf(Timestamp < incident_start AND StatusCode = 'Error')
AS baseline_errors,
    countIf(Timestamp >= incident_start) AS incident_requests,
    any(TraceId) AS example_trace_id
FROM default.otel_traces
WHERE ServiceName = {service:String}
  AND SpanKind = 'Server'
  AND Timestamp >= incident_start - toIntervalMinute({incident_minutes:UInt32})
GROUP BY endpoint, release, status_message
HAVING incident_requests > 0
ORDER BY incident_errors - baseline_errors DESC, incident_requests DESC
LIMIT 20;
```

Ask what your own tool does with forty attributes at 3am. If it needs you to choose one first, it has not answered.

> **Next problem**
>
> **A delta covers the attributes. Ten thousand log messages arrive too, and nobody reads that many.**

What do ten thousand log messages actually say?

Nobody reads them all. The one that matters hides among the ones that always appear.

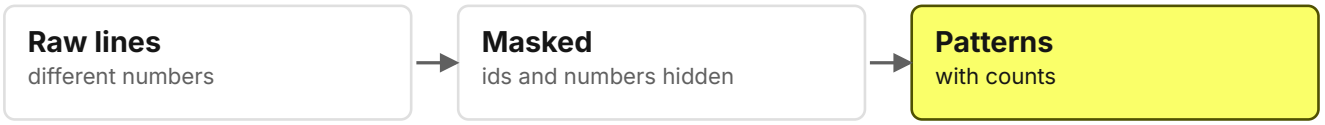


Figure 24 Hiding the parts that vary makes different lines count as one message.

So we hide the parts that vary, like numbers and ids. Lines that were different now count as the same message, and thousands become a handful.

```
SELECT
    replaceRegexpAll(
        replaceRegexpAll(replaceRegexpAll(Body, '\\[[^\\]]*\\]', '[ts]'), '[0-
9a-f]{16,}|\\S+@\\S+', '<id>'),
        '\\d+(\\.\\d+)?', 'N'
    ) AS pattern,
    count() AS events,
    groupUniqArray(3)(SeverityText) AS severities,
    min(Timestamp) AS first_seen,
    max(Timestamp) AS last_seen,
    any(Body) AS example
FROM default.otel_logs
WHERE ServiceName = {service:String} AND Timestamp > now() -
toIntervalMinute({minutes:UInt32})
GROUP BY pattern
ORDER BY events DESC
LIMIT 25;
```

The biggest group is usually the same noise as in a healthy hour. Sort by first seen before you sort by count.

> **Next problem**
>
> **Patterns show what was said, not where the time went. One request took nine seconds.**

Where did the nine seconds go?

A request took nine seconds. The service that answered says it spent two hundred milliseconds. Somebody downstream held the rest, and no log line says who.

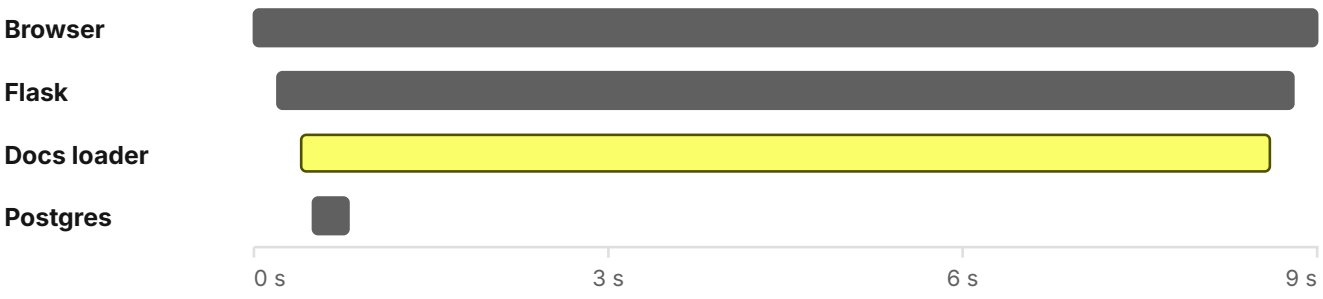


Figure 25 The marked bar holds nearly all the time, so the slowness is not in the layers above it.

So we draw the request as a waterfall, one bar per step against time. A slow parent with one slow child reads very differently from one with fifty fast ones. Only the waterfall shows which.

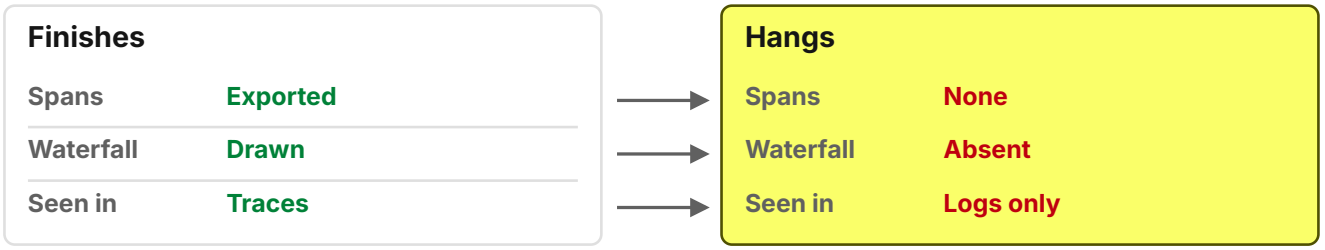


Figure 26 A step that never ends exports nothing. The gap is the clue.

The lab query returns each step with its start and length, which is all a waterfall needs.

```
SELECT
    SpanId AS span_id,
    ParentSpanId AS parent_span_id,
    ServiceName AS service,
    SpanName AS span,
    SpanKind AS kind,
    round(dateDiff('microsecond', min(Timestamp) OVER (), Timestamp) / 1e3, 1)
AS start_offset_ms,
    round(Duration / 1e6, 1) AS duration_ms,
    StatusCode AS status,
    StatusMessage AS status_message,
    SpanAttributes['db.statement'] AS db_statement
FROM default.otel_traces
WHERE TraceId = {trace_id:String}
ORDER BY Timestamp
LIMIT 200;
```

A step that never finishes leaves nothing to draw. So teach the agent to ask what should be there and is not.

> **Next problem**
>
> **Now we can find the cause. Next we must decide which tools the agent gets.**

Module four

What may the agent actually do?

This chapter decides whether the rest is safe to run on a real system.

Who decides which tools the agent gets?

Someone adds a helpful new tool on a Friday. It reads logs and, six lines further down, it can also restart a container.

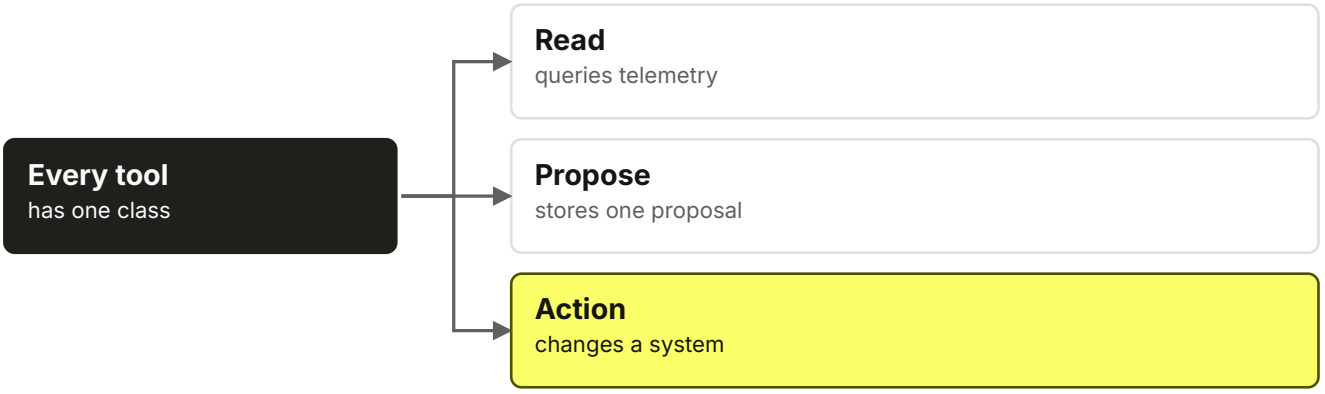


Figure 27 Most tools only read. One proposes, and exactly one can act.

A sentence in a prompt is a request. A tool that does not exist is a limit. So every tool must declare a class in code: it reads, it proposes, or it acts.

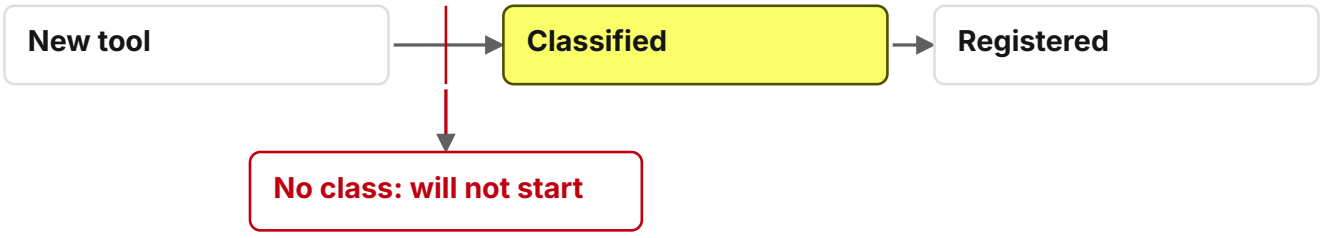


Figure 28 The forgotten case fails on the day it is written, not later in production.

A tool with no class stops the server from starting, so the mistake shows up the day it is made.

```
"get_remediation_status": ToolClass.READ,
"propose_remediation": ToolClass.PROPOSE,
"execute_remediation": ToolClass.ACTION,
```

The list lives next to the server, so a reviewer sees it in the same change as the tool.

> **Next problem**
>
> **Even a read tool can run any query, and some queries hurt a database.**

What stops a query hurting the database?

Handing a model a SQL tool sounds alarming. It is, until you look at the account the query runs as.

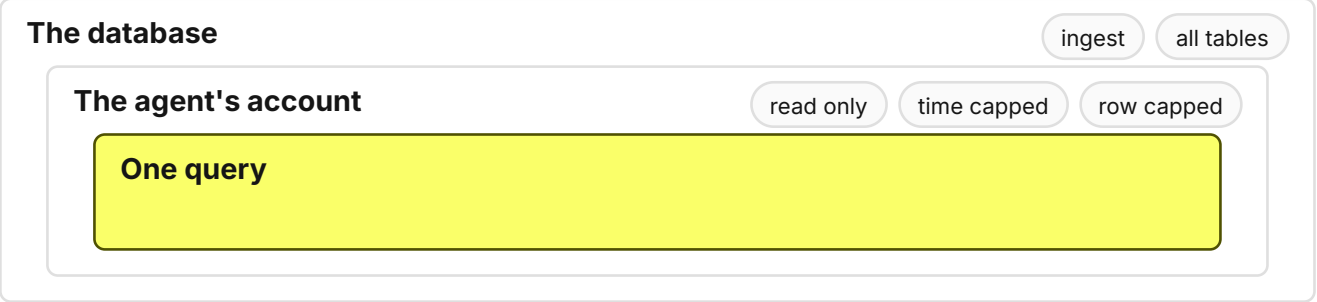


Figure 29 Each ring reaches only what sits inside the one around it. The account draws them, not the model's manners.

So the account is read only, with a time limit and a row limit. The worst a bad query can do is fail. Without limits, it starves the ingest pipeline and the incident gets worse.

```
<readonly>2</readonly>
<max_execution_time>10</max_execution_time>
<max_memory_usage>1000000000</max_memory_usage>
<max_result_rows>10000</max_result_rows>
```

Look up the account your own agent connects as. Read its limits before you read its prompt.

> **Next problem**
>
> **The database is safe, but the story can still be wrong. The agent may cite a trace that does not exist.**

What if the agent cites something that does not exist?

The root cause reads perfectly. It names a release, two trace ids and a service, and one of those trace ids is not real.

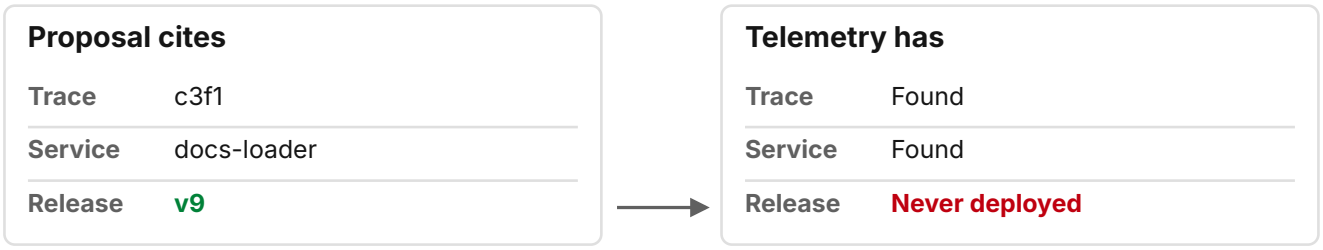


Figure 30 A fluent claim about a release nobody shipped stops here.

A model can write a fluent cause for an incident it does not understand, and it reads just like a real one. So plain code looks up every id before the proposal is stored.

```
if not trace_ids:
    raise Ungrounded("cite at least one trace id returned by a tool")
if not self.telemetry.service_seen(service):
    raise Ungrounded(f"service {service!r} has no telemetry in {window}")
if release and not self.telemetry.release_seen(service, release):
    raise Ungrounded(f"release {release!r} of {service} has no telemetry
in {window}")
missing = self.telemetry.missing_trace_ids(trace_ids)
if missing:
    raise Ungrounded(f"these trace ids have no spans in {window}:
{', '.join(missing)}")
```

Because the check runs first, a tired person never has to be the one who spots it.

Next problem

A true proposal still has to become an action. Text must never turn into a command.

How do we stop text becoming a command?

A log line the agent reads says, in the middle of an error, to run a cleanup command. The agent is helpful, and the runner does what it is asked.

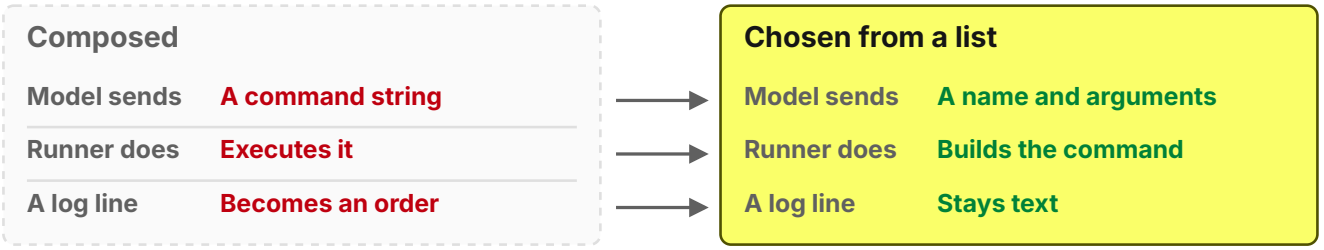


Figure 31 Who builds the command decides whether text the agent read can become an order.

Anyone who can write to your logs can write to the agent's context. So the runner takes a name from a list, plus arguments, and builds the command itself. It never runs a string.

```
def build_command(action: str, target: str, params: dict[str, str],
project_dir: str) -> tuple[list, dict]:
    validate_action(action, target, params)
    base = ["docker", "compose", "--project-directory", project_dir, "-f", f"
{project_dir}/{COMPOSE_FILE}"]
    if action == "rollback_release":
        # Redeploying the previous release is `up` on the previous image tag,
as a deploy system would.
        return [*base, "up", "-d", "--no-deps", "--wait", target],
{"APP_RELEASE": params["release"]}
    return [*base, "restart", target], {}
```

Find any place in your own automation where an outside string ends up in a shell. That is the gap.

> **Next problem**
>
> **The list limits what can run. A person's approval must also be a stored fact.**

Module five

Where does the person come in?

Everyone agrees a person should approve production changes. Here is what that has to become in code.

What makes approval a stored fact?

Someone tells the agent in a chat window that the fix is fine and to go ahead. Nothing in the system recorded that anyone agreed.

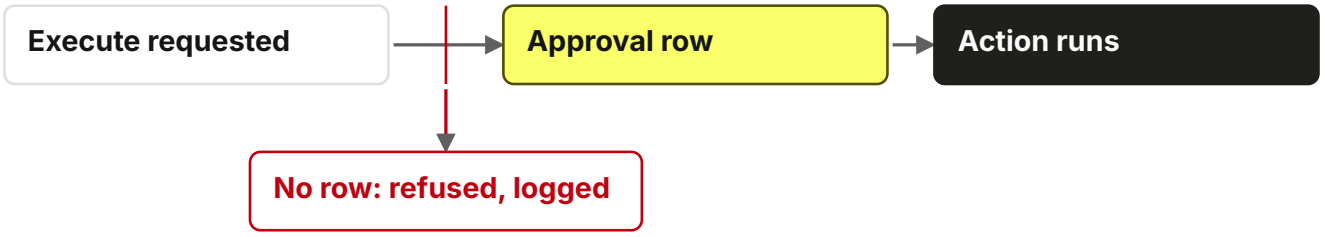


Figure 32 The server answers, whatever the model was asked.

A request to act is only a request. So the tool that acts looks for a stored row that names a person and one proposal. No row, no action.

```
if current.state != "approved":
    self.store.append(
        Event(self.clock(), current.incident_id, action_id,
"remediation_refused", executed_by,
              {"state": current.state})
    )  # fmt: skip
    raise ApprovalRequired(
        f"{action_id} is {current.state}; a human must approve it at
/actions/{action_id} first"
    )
result = self.runner.run(current.action, current.target, current.params)
```

The refusal is recorded too, so an attempt to skip the person shows up in the same log as everything else.

> **Next problem**
>
> **A stored fact can still be edited. We need one history that stays honest.**

How do we keep one honest history?

Two people disagree about who approved the rollback. One remembers Priya, and the ticket names someone else.

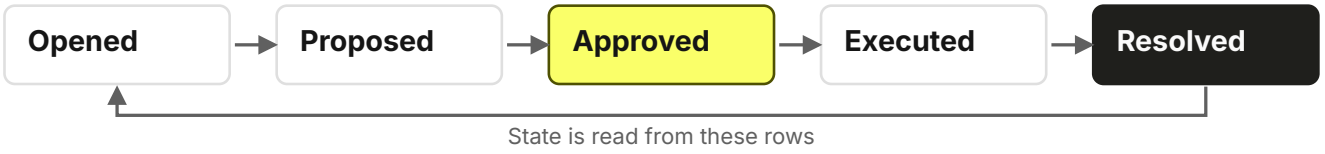


Figure 33 Rows are only ever added, so the past cannot be rewritten.

When a status field is changed in place, the past is whatever the last writer decided. So every step adds a new row, and nothing is ever changed. The state is read from the rows.

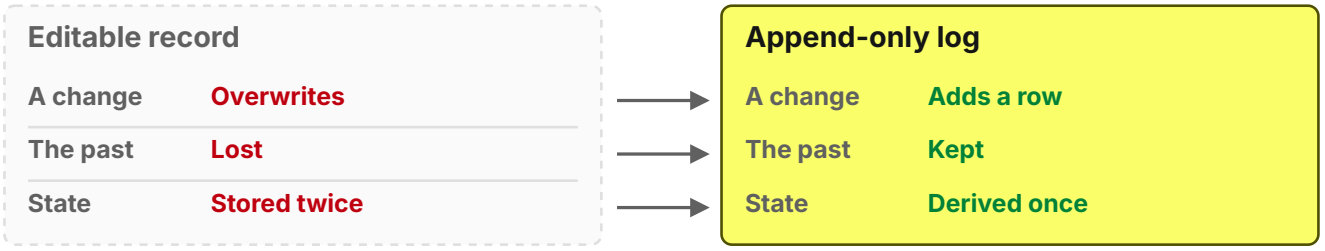


Figure 34 A row you can update is a history that can disagree with itself.

Ask whether your own tools give one answer to who approved what, or two.

> **Next problem**
>
> **The history says what ran. But a command can finish cleanly while signups still fail, so the incident is not really over.**

When is the incident really over?

The rollback command printed its last line and exited zero. Everyone relaxed, and the signup page was still throwing errors.

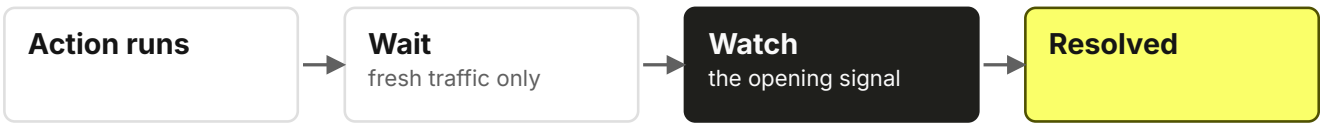


Figure 35 The signal that opened the incident is the one that closes it.

An exit code says only that the command finished. So we let fresh traffic arrive, then check the same signal that raised the alarm.

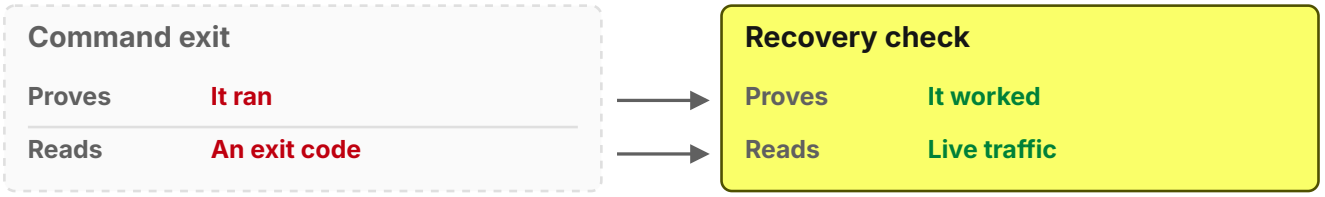


Figure 36 Exit zero proves a command ran. It does not prove signups work.

```
if now - executed_at < RECOVERY_WINDOW:
    continue
healthy, evidence = self._healthy(incident)
timed_out = now - executed_at > timedelta(minutes=5)
if healthy or timed_out:
    self.remediations.record_verification(action.action_id,
passed=healthy, evidence=evidence)
```

Verify by running it, never by reading it. That is the rule for tests, applied to incidents.

> **Next problem**
>
> **A person decides and the system checks. Some actions deserve more trust in the model than others.**

Should we trust the model with everything?

People ask whether you trust the model. It is the wrong question, because the answer for a restart is not the answer for a database.



Figure 37 The lab sits at level two for every action. Moving one up is a choice about damage.

Google's operations practice describes levels from fully manual to fully automatic. So we place each action on the levels by itself, and never the whole agent.

Action	Damage	Undone
Restart a container	Seconds of errors	Nothing to undo
Roll back a release	Withdraws new code	Deploy it again
Change a database	**Data can be lost**	**Often cannot**

Figure 38 The first two rows exist in the lab. The database change is only an illustration.

Ask which actions you would let a new engineer run alone in their first week. Those are your candidates for level three.

> **Next problem**
>
> **Every level assumes honest data. Someone can write to the logs on purpose.**

Module six

How can this go wrong?

OWASP listed the threats that belong to agents, not models, in December 2025. Several apply to anything that reads production data and can act.

What if someone writes to the logs on purpose?

A user signs up with a name that is a sentence aimed at whoever reads the logs. The agent reads the logs.

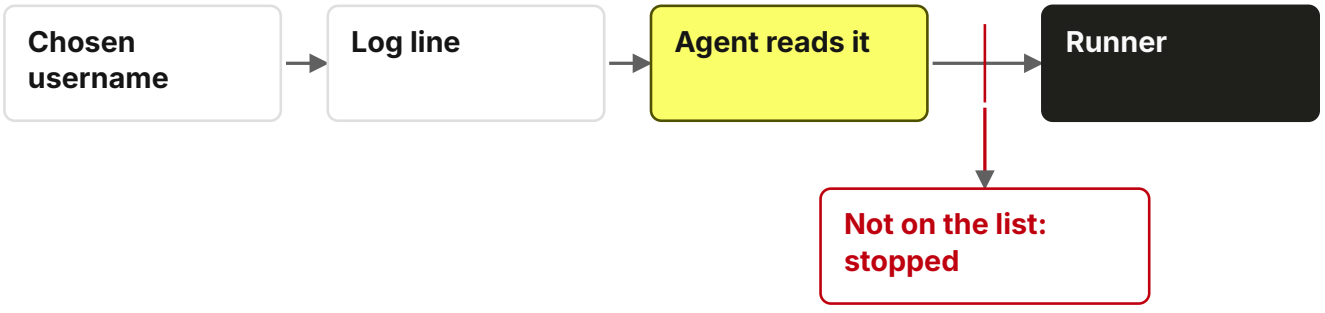


Figure 39 The log line reaches the agent. The list stops it going any further.

An agent cannot reliably tell an instruction from plain content. OWASP calls this goal hijack, and lists it first. Anyone who can make your system log a string can speak to

the agent.

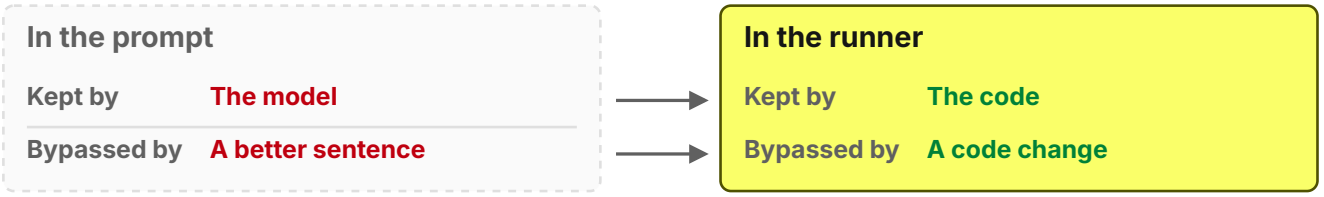


Figure 40 A sentence can be argued with. A list in code cannot.

So the fix is not a better instruction. The tool that acts takes a name from a list, never a command from text.

> **Next problem**
>
> **A log line lasts one step. A false sentence can also stick in memory.**

What if a false sentence sticks in memory?

One wrong sentence gets into the summary of the incident. Twenty minutes later, three steps reason from it as if it were measured.

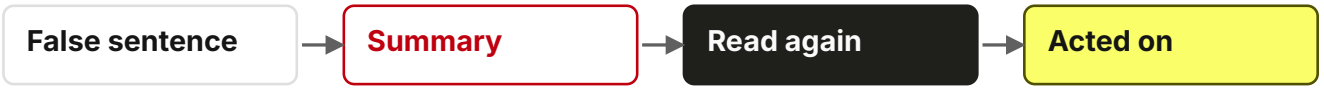


Figure 41 What the agent writes down is what it reads next.

This is the same attack, stretched over time. Whatever the agent carries between steps is something it will trust.

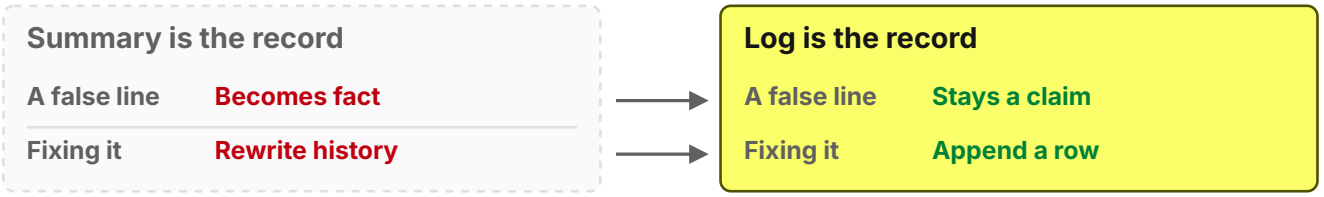


Figure 42 With the log as the record, a wrong sentence stays wrong.

So we read state from the append-only log. A wrong sentence then stays a claim and never becomes the record.

> **Next problem**
>
> **The agent may be safe. The tired person who approves is the next weak spot.**

What about the tired person who approves?

It is four in the morning and the page says Approve or Reject. The last nine proposals were right.

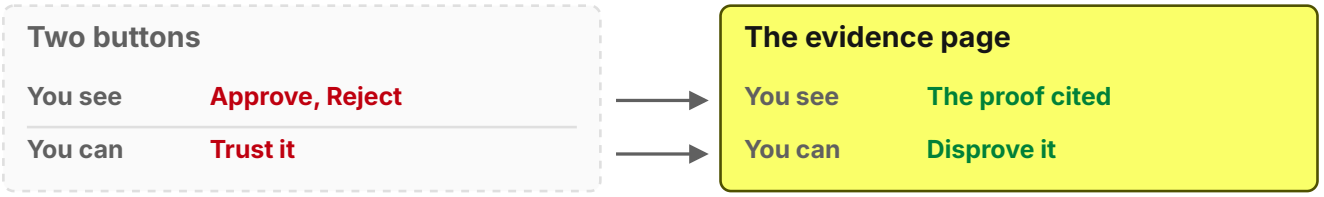


Figure 43 What is on the page decides whether the approval means anything.

OWASP calls this trust exploitation. A person approves because approving is what they do. The button is not the safeguard. The proof beside it is.

On the page	**Lets you check**
Evidence	The cause is real
Blast radius	The cost is acceptable
Telemetry link	Nothing was invented

Figure 44 The test for your own page: what could a reader disprove in a minute?

Open your own approval screen and time how long it takes to prove the proposal wrong.

> **Next problem**
>
> **Good proof helps the person. But convenience is risky, because it widens the agent's reach.**

Why is convenience risky?

Mounting the container socket was the quickest way to let one action restart one service. Nobody wrote down what else it could do.

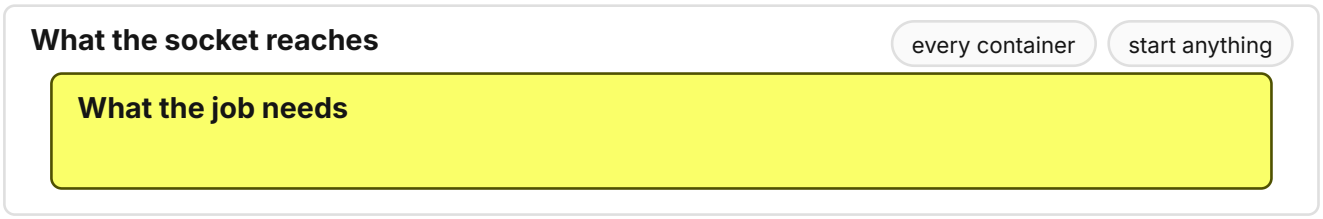


Figure 45 Access wider than the job is access the agent has.

Access that grows past the task is access the agent has. A socket that can restart one container can start any container.

Control	Answers
Least privilege	What can it reach?
Circuit breaker	When does it stop?
Dry run	What would happen?
No ambient access	Who lent it that?

Figure 46 The same controls, in the words of the operations team.

This lab makes that trade on purpose, and the hardening chapter exists because of it. Copy the design without the hardening and it goes wrong.

> **Next problem**
>
> **Each fix is a guess until it is measured. We cannot know if the agent got better.**

How do we know the agent got better?

Someone improves the prompt on Tuesday. On Wednesday nobody can say whether the agent is better, because nothing was measured.

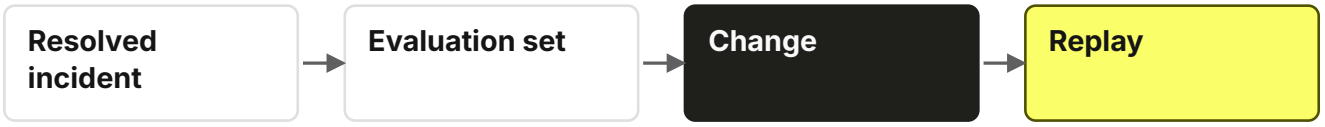


Figure 47 Every resolved incident becomes a test for the next change.

So we keep every incident we resolve, with the cause it turned out to have. Before any change to the prompt, model or tools, we replay them. This is an evaluation set.

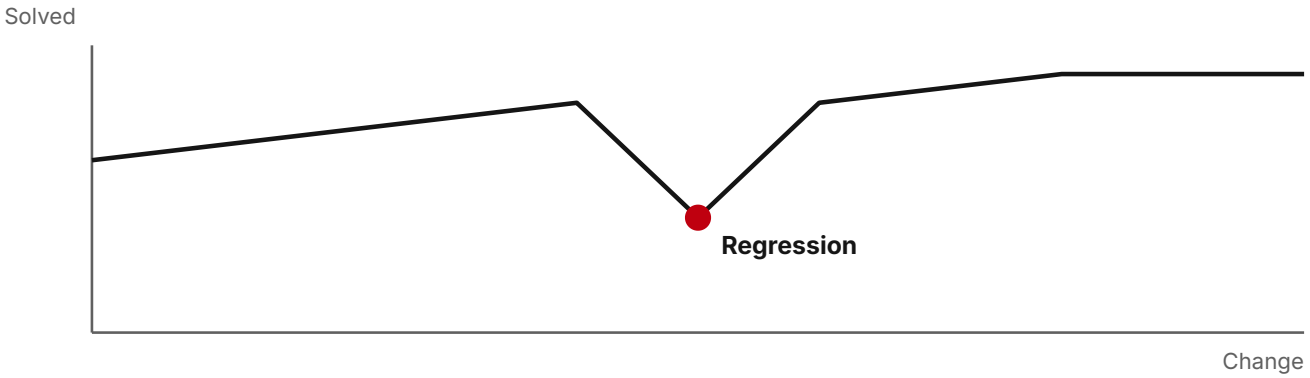


Figure 48 The dip is the point. Without the replay, an incident would have found it.

Start today. Every resolved incident is the first test in one.

> **Next problem**
>
> **You have the whole loop. Some ideas are published, and we should say where they come from.**

Grounding

Where do these ideas come from?

Knowing what is published and what is one lab's choice lets you argue with the second kind.

Which ideas are published, and where did they come from?

None of this vocabulary was invented here. Indicator, objective and error budget come from Implementing SLOs (https://sre.google/workbook/implementing-slos/) in Google's Site Reliability Workbook. Burn rate comes from Alerting on SLOs (https://sre.google/workbook/alerting-on-slos/), in the same book.

The three signals are a standard too. OpenTelemetry (https://opentelemetry.io/docs/concepts/signals/) defines traces, metrics and logs, so these queries work on any vendor's storage.

> **Next problem**
> **Those words are old. The guardrails have names too.**

What names do the guardrails have?

Google's guidance on AI engineering for reliable operations (https://sre.google/resources/practices-and-processes/ai-engineering-reliable-operations/) names the controls this lab uses. They are least privilege, no ambient access, circuit breakers, dry runs and safe defaults. It also gives the autonomy levels.

Read their list, then read what the lab enforces. The gap you find is the work left before this runs on your own systems.

> **Next problem**
> **Guardrails answer risks. Those risks are written down too.**

Where are the risks written down?

OWASP published its Top 10 for Agentic Applications (https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/) in December 2025. It names risks that belong to agents, not to models.

The hardening chapter uses their names. That lets a security reviewer match this design to a list they already know.

> **Next problem**
> **Published names cover most of it. The rest is this lab's own choice.**

What is only this lab's own choice?

Some choices are only ours. One is the three sigma band, with a fixed ceiling under it. Another is dropping past incidents from the baseline. The last is holding every action at level two.

They are defensible. They are not standards. Treat them like any design in a repository you did not write. The published material shows the options, and the lab shows what one set of answers costs.

Recap

The words this workshop uses

If you take one page away, take this one. Each word is explained in the chapter it belongs to.

Term	What it means here
Wide event	One record per request, with every fact you might group by later
Cardinality	How many different values a fact can take. More values cost more
Sampling	Keeping only some traces, which loses the rare ones you need
Indicator	What a user feels, as good events over valid events
Objective	The target that indicator must hit, over a stated time
Error budget	The failure the objective allows, as an amount to spend
Burn rate	How fast the budget is being spent
Baseline	The stretch of time you call healthy
Three sigma	A band around the baseline that ordinary minutes stay inside
Event delta	The facts that differ between a healthy window and a failing one
Trace waterfall	One request's steps against time, showing where it waited
Tool class	What a tool may do, declared in code, or it cannot register
Grounding	Checking that every id a proposal cites is real
Approval event	The stored record of a person saying yes
Recovery check	Watching the opening signal on fresh traffic before closing
Autonomy level	How much runs without a person, chosen per action
Goal hijack	An instruction planted where the agent will read it
Evaluation set	Past incidents with known answers, replayed after every change

The facilitator guide carries the timing and the commands for the day, and it cites these ideas by name rather than explaining them again. Everything described here is exercised by `make e2e` against a real model, and scored by `make score`.